

Tech Challenge – Fase 1 – 15SOAT

1. Nome do Grupo: SPRING-JAVA-SOAT

2. Participantes:

Gabriel Sartoretto - RM372178

Henrique Dellagnesi Azevedo - RM372407

José Manoel - RM37365

Karen Lima Barcelos - RM374165

Lenilson Florencio – RM370864

3. Link da documentação:

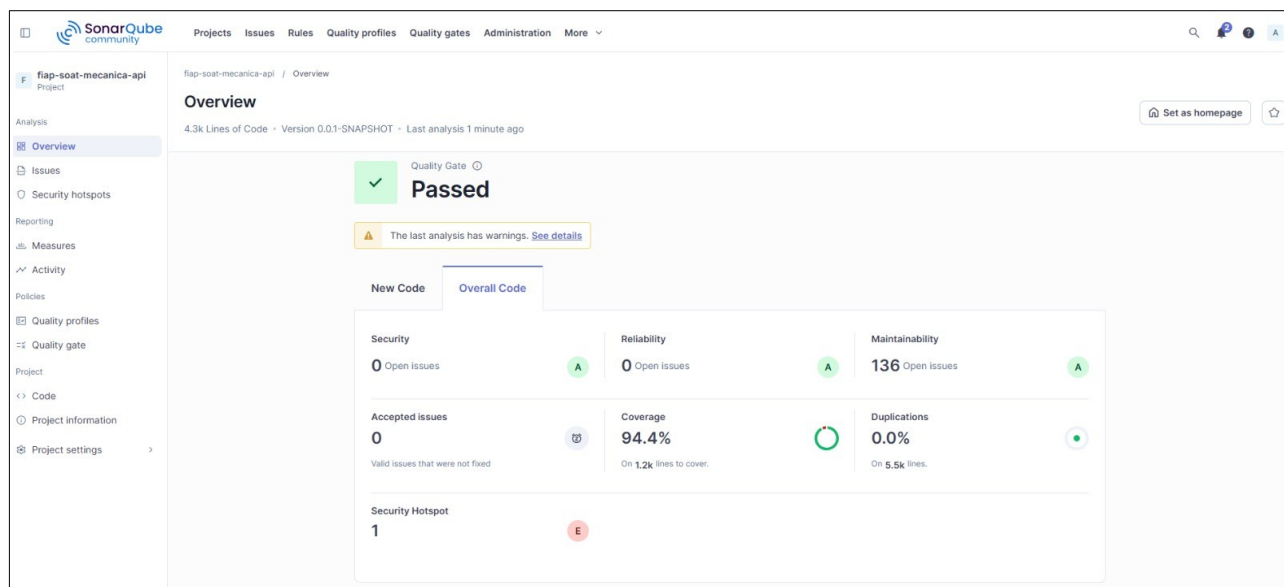
DDD – Miro: https://miro.com/app/board/uXjVHaE9VQI=/?share_link_id=966474134117

BD – Figma: <https://www.figma.com/board/fcHwkB5QjBZdpmoCBECiHu/Tech-Challenge?node-id=0-1&t=VcSeWI71VFkwK776-1>

4. Link do Repositório:

Github: <https://github.com/gabriel-sartoretto/fiap-soat-mecanica-api>

5. Relatório com análise de vulnerabilidades encontradas no sistema:



6. Estudo técnico para definição de banco de dados e stack

Teorema CAP, análise do domínio da oficina, matriz de decisão, justificativa da stack e decisão final

6-1. Explicação aprofundada do Teorema CAP

O Teorema CAP é um modelo teórico para analisar sistemas distribuídos. Ele afirma que, na presença de partição de rede, um sistema distribuído não consegue maximizar simultaneamente Consistency, Availability e Partition Tolerance. Em termos práticos, quando ocorre falha de comunicação entre nós, é necessário escolher entre priorizar consistência imediata dos dados ou disponibilidade contínua das respostas.

Os três pilares podem ser entendidos assim:

Consistency (C): todos os nós veem o mesmo dado após uma escrita.

Availability (A): o sistema sempre responde, mesmo que com dados desatualizados.

Partition Tolerance (P): o sistema continua funcionando mesmo com falhas de rede.

Na prática, o CAP mostra que, durante partições, precisamos escolher entre consistência e disponibilidade.

Neste projeto, como estamos trabalhando com um monólito, o CAP é mais uma base conceitual do que um fator decisivo direto.

6-2. Análise do domínio da oficina

O sistema envolve:

- ordens de serviço com fluxo de status
- aprovação de orçamento
- controle de estoque
- histórico de clientes e veículos
- execução de serviços

Principais características:

- Orientado a estados
- Transacional
- Relacional
- Baixa tolerância a inconsistência

Conclusão: o domínio exige consistência forte.

6-3. Matriz de decisão do banco de dados

PostgreSQL:

- excelente consistência
- forte modelo relacional
- ideal para transações

MySQL:

- viável, porém menos robusto

Decisão: PostgreSQL

6-4. Justificativa arquitetural

O Teorema CAP foi utilizado como base conceitual para a análise de persistência. Como o domínio exige consistência forte em operações de criação e atualização de ordens de serviço, aprovação de orçamento e controle de estoque, não seria adequado priorizar uma abordagem orientada à disponibilidade eventual. Embora o sistema da Fase 1 seja monolítico e, portanto, não esteja plenamente inserido no cenário clássico de trade-offs entre nós distribuídos, o CAP ajuda a justificar a opção por um banco relacional transacional. Por essa razão, foi escolhido o PostgreSQL, que oferece forte consistência transacional, constraints, integridade referencial e modelagem adequada para relações entre clientes, veículos, serviços, peças e ordens de serviço. Além disso, o PostgreSQL tem ciclo de suporte claro: cada major version recebe suporte por cerca de 5 anos; atualmente há versões suportadas como 14, 15, 16, 17 e 18.

6-5. Decisão final

Stack definida:

- Java 21
- Spring Boot 3.5.x
- PostgreSQL 17
- Spring Data JPA
- Spring Security + JWT
- Flyway
- Swagger
- Testes automatizados
- Lombok controlado